Technical Specification: caRtola Cartola FC Decision Platform

Status: Draft
Version: 0.1
Created: 2026-05-16
Last Updated: 2026-05-16
Owner: caRtola maintainers
Reviewers: Unknown

1. Summary

caRtola is a Python and Kedro based Cartola FC data science platform for historical analysis, leakage-safe backtesting, model research, live squad recommendation, and guarded submission planning. This specification defines the project-level product and technical contract that AgileForge can use as the canonical management spec for future planning. It consolidates repository behavior that is currently spread across the README, feature design specs, operational scripts, and repository instructions.

2. Problem Statement

Cartola lineup decisions are path-dependent, deadline-sensitive, and vulnerable to data leakage. The repository has evolved from historical data collection and exploratory modeling into a decision platform with multiple operational modes, but there is no single project-level spec that states what the platform must do, what evidence is acceptable for promotion, and which capabilities remain research-only.

Observed facts:

- Historical Cartola data lives under `data/01_raw/` and reporting artifacts are written under `data/08_reporting/`.
- The active Python package is `src/cartola`; operational commands live under `scripts/`.
- Backtests and experiments now use moving-budget semantics and the `cartola_standard_2026_v1` scoring contract.
- Live recommendations default to `xgboost_depth2_l2_heavy + ppg_xg`, `fixture_mode=none`, and `matchup_context_mode=none`.
- Real authenticated squad submission is intentionally disabled in Phase 1; the current submission flow only produces a sanitized submission plan.

Assumptions:

- AgileForge will ingest this file through `agileforge project create --spec-file specs/app.md` or a future spec update command.
- The initial AgileForge management scope is product and technical governance, not immediate implementation ticket generation.

3. Goals And Non-Goals

Goals

- Provide one canonical project-level spec for caRtola that is suitable for AgileForge project management.
- Preserve leakage-safe historical evaluation rules for backtests, experiments, and replay recommendations.
- Define live recommendation behavior, artifact requirements, and approved production defaults.
- Separate production recommendation evidence from exploratory research, policy simulation, oracle discovery, and diagnostics.

- Define the guarded submission-planning workflow while explicitly preventing real authenticated submission in Phase 1.
- Make quality, security, privacy, performance, reliability, and migration expectations reviewable.

Non-Goals

- Do not define implementation tasks, sprint tickets, commit plans, or code changes.
- Do not authorize changing live model defaults from research artifacts alone.
- Do not enable real authenticated Cartola squad submission.
- Do not require browser automation as an operational interface.
- Do not require fixed-budget historical backtesting compatibility for new evidence.
- Do not require all old notebooks or R scripts to be migrated into the Python backtesting system.
- Do not treat exploratory reconstructed fixtures as strict no-leakage proof.

4. Users And Stakeholders

- **Primary users:** caRtola maintainers and operators who run historical experiments, audits, and live recommendation workflows.
- **Secondary users:** data science contributors who add feature packs, models, diagnostics, and reports.
- **Operational stakeholders:** the person making weekly Cartola lineup decisions before market lock.
- **External systems:** Cartola market/status and athlete APIs, TheSportsDB fixture source, FootyStats local data exports, OpenMP/BLAS native runtimes, optional MLflow tracking, GitHub Actions, and AgileForge project management.

5. Current State

The repository is a Python `uv` project targeting Python `3.13.12`, with the main package under `src/cartola` and tests under `src/tests`. Kedro remains part of the project shape, while most current operational work is exposed through scripts and `cartola.backtesting` modules.

Current delivered capabilities include:

- Historical season loading and compatibility audits across local Cartola raw seasons.
- Walk-forward backtesting with moving-budget semantics.
- Captain-aware optimizer using all official Cartola formations and a non-tecnico `1.5x` captain multiplier.
- Fixture modes: `none`, `exploratory`, and `strict` in historical contexts, with live strict fixture support only when pre-lock evidence exists.
- FootyStats feature modes: `none`, `ppg`, and `ppg_xg`.
- Matchup context mode `cartola_matchup_v1` for controlled research and strict live opt-in.
- Model-feature experiment runners, policy simulations, oracle discovery, EBM diagnostics, H004/H005 research, fixed-blend diagnostics, Ridge promotion decisions, and XGBoost Optuna tuning.
- One-command live recommendation through `scripts/run_live_round.py`.
- Phase 1 submission planning through `scripts/submit_recommended_squad.py`, with real submit disabled.

Primary reference files:

- `README.md`
- `AGENTS.md`
- `roadmap.md`
- `pyproject.toml`
- `docs/superpowers/specs/`
- `scripts/run_live_round.py`
- `scripts/recommend_squad.py`
- `scripts/submit_recommended_squad.py`

- `src/cartola/backtesting/`

6. Proposed Specification

6.1 Functional Requirements

ID	Requirement	Acceptance Criteria	Priority
FR-001	The project must load local historical Cartola market data for supported seasons.	Compatibility audit reports each requested season as loadable, irregular, partial, or incompatible with explicit reasons.	Must
FR-002	Historical backtests must use moving-budget semantics.	Each strategy records `budget_policy=moving`, `initial_budget`, per-round budget before/after, budget delta from selected `variacao`, final budget, min budget, and max drawdown.	Must
FR-003	Historical backtests must prevent future-round leakage.	For target round `N`, model training uses only rows from rounds `< N`; candidate optimization uses only rows from round `N`; reports record the evidence boundary.	Must
FR-004	Backtests and recommendations must use the `cartola_standard_2026_v1` scoring contract.	Reports identify the scoring contract, evaluate all official formations, select exactly one non-tecnico captain, and expose captain-adjusted totals.	Must
FR-005	Live recommendation must capture or validate the open market round before recommendation.	`scripts/run_live_round.py` either captures a fresh live market snapshot or validates an existing live capture according to `capture_policy=fresh	missing
FR-006	Live recommendation artifacts must be archived without overwriting earlier runs.	Each one-command live run writes under `data/08_reporting/recommendations/{season}/round-{round}/live/runs/run_started_at=.../`.	Must
FR-007	Live recommendations must use the approved default profile unless the operator explicitly overrides it.	Default CLI values are `model_id=xgboost_depth2_l2_heavy`, `footystats_mode=ppg_xg`, `fixture_mode=none`, and `matchup_context_mode=none`.	Must
FR-008	Strict fixture live recommendations must fail closed when strict evidence is missing.	`fixture_mode=strict` requires canonical strict fixture CSV and manifest files for the target season and round; missing or invalid evidence raises before recommendation.	Must
FR-009	Exploratory fixture evidence must not be represented as strict no-leakage proof.	Reports and metadata distinguish `fixture_mode=exploratory` from `fixture_mode=strict`; promotion decisions cannot use exploratory-only evidence as strict validation.	Must
FR-010	FootyStats joins must be auditable and leakage-safe.	Metadata records FootyStats mode, league slug, source paths or hashes, join diagnostics, and excludes unsafe post-match fields from pre-round features.	Must
FR-011	Matchup context must remain opt-in outside approved	`matchup_context_mode=cartola_matchup_v1` is unavailable unless a compatible fixture mode is selected; live	Must

ID	Requirement	Acceptance Criteria	Priority
	defaults.	defaults remain `none`.	
FR-012	Experiments must fail closed on comparability drift.	Model-feature experiment reports include comparability signatures for source data, candidate pools, optimizer status, budget policy, fixture mode, feature pack, and model identity.	Must
FR-013	Promotion decisions must be based on frozen comparable artifacts.	A model, feature pack, blend, policy, or tuned candidate changes live defaults only after a dedicated decision artifact passes stated guardrails.	Must
FR-014	Research diagnostics must be marked as non-production evidence.	Oracle discovery, EBM diagnostics, policy simulation, and hypothesis diagnostics record discovery or research-only status and do not update live defaults.	Must
FR-015	Recommendation outputs must separate raw per-player predictions from captain-adjusted totals.	Selected-player files keep `predicted_points` raw; summaries and round totals use `predicted_points_with_captain` and `actual_points_with_captain` when available.	Must
FR-016	Replay recommendations must attach actual results only after prediction and optimization.	Replay mode can include actual points in outputs, but target-round actuals are not used before candidate scoring or optimization.	Must
FR-017	Live recommendation outputs must suppress finalized target-round outcomes.	Live mode rejects finalized target-round evidence unless an explicit debug flag is used, and still omits actual/scout outcome columns from live outputs.	Must
FR-018	Phase 1 submission must produce a sanitized submission plan from an existing live recommendation artifact.	`scripts/submit_recommended_squad.py --recommendation-path ...` validates the artifact and current market, then writes `submission_plan.json` and `submission_result.json` under a unique attempt directory.	Must
FR-019	Real authenticated submission must be disabled in Phase 1.	Any `--confirm-submit` invocation exits nonzero with `CONTRACT_UNVERIFIED` before loading `.env`, reading `CARTOLA_GLB_TOKEN`, creating an authenticated client, or constructing a POST request.	Must
FR-020	Repository quality gates must remain reproducible.	`uv sync --locked --dev` and `uv run --frozen pyrepo-check --all` reproduce the GitHub Actions quality gate.	Must

6.2 User Scenarios

```
Scenario: Run a historical no-fixture backtest
  Given local raw Cartola data exists for the requested season
  When the operator runs the backtest CLI with fixture_mode none
  Then the system evaluates target rounds sequentially using moving budgets
  And writes round, selected-player, metadata, summary, and chart artifacts
```

```
Scenario: Generate a live recommendation for the open market round
  Given the Cartola market is open for the current season
  When the operator runs scripts/run_live_round.py with capture_policy fresh
```

```
  Then the system captures the open market, recommends a squad for that captured
round, and archives the run with metadata linking the capture and recommendation
```

```
Scenario: Reject strict live matchup recommendation without evidence
  Given strict fixture CSV or manifest evidence is missing for the live target
round
  When the operator requests fixture_mode strict
  Then the command fails before recommendation
  And does not fall back to exploratory or no-fixture mode
```

```
Scenario: Build a submission plan without real submit
  Given a reviewed live recommendation run exists
  When the operator runs scripts/submit_recommended_squad.py with the
recommendation path
  Then the system validates the artifact and writes a sanitized submission plan
  And any confirm-submit attempt fails with CONTRACT_UNVERIFIED before auth
handling
```

6.3 Data And State

Core entities:

- **Raw Cartola market round:** CSV rows under `data/01_raw/{season}/rodada-{round}.csv`; live captures have a paired `.capture.json` manifest.
- **Fixture evidence:** exploratory fixture CSVs under `data/01_raw/fixtures/{season}/`; strict fixture CSVs and manifests under `data/01_raw/fixtures_strict/{season}/`.
- **Feature frame:** leakage-safe per-round model input built from visible historical data and optional FootyStats or matchup context.
- **Candidate prediction:** per-athlete target-round prediction with model, feature-pack, fixture, and candidate-pool metadata.
- **Optimized squad:** selected 12-row squad with exactly one tecnico, 11 non-tecnico players, one non-tecnico captain, official formation, budget usage, and optimizer status.
- **Backtest run:** multi-round historical replay with moving-budget path per strategy.
- **Experiment run:** matrix of comparable child backtests with ranked summaries, prediction metrics, comparability report, and reports.
- **Live recommendation run:** archived output directory containing recommended squad, candidate predictions, summary, run metadata, and live workflow metadata.
- **Submission attempt:** sanitized audit directory beside a live recommendation run containing submission plan and result files.

Important invariants:

- Historical multi-round workflows update budget only after selecting and scoring a squad.
- Live single-round workflows treat `--budget` as current available budget for that round only.
- Old fixed-budget artifacts are not comparable with moving-budget artifacts.
- Strict fixture mode never silently falls back to exploratory or no-fixture mode.
- Real authenticated submission cannot be enabled without a separate accepted Phase 2 spec.

6.4 Interfaces And Integrations

Primary local commands:

```
uv run --frozen python -m cartola.backtesting.cli --season 2025 --start-round 5 --
budget 100 --fixture-mode none
uv run --frozen python scripts/run_live_round.py --season 2026 --budget 100 --
current-year 2026
uv run --frozen python scripts/recommend_squad.py --season 2026 --target-round 10 -
-mode replay --budget 100 --current-year 2026
uv run --frozen python scripts/capture_market_round.py --season 2026 --auto --
current-year 2026
uv run --frozen python scripts/capture_strict_round_fixture.py --season 2026 --auto
--current-year 2026
uv run --frozen python scripts/run_model_experiments.py --group production-parity -
-seasons 2021,2022,2023,2024,2025 --start-round 5 --budget 100 --current-year 2026
uv run --frozen python scripts/submit_recommended_squad.py --recommendation-path
data/08_reporting/recommendations/2026/round-16/live/runs/run_started_at=...
```

External integrations:

- Cartola public market APIs for live market status, athlete data, and fixture evidence.
- TheSportsDB for fixture imports and exploratory historical fixture reconstruction.
- FootyStats local CSV data for pre-match PPG and xG features.
- Optional MLflow local tracking through an explicit tracker adapter.
- XGBoost native runtime, including OpenMP availability on macOS.
- AgileForge CLI for project management using this spec file.

Authentication:

- Phase 1 submission planning does not require authentication.
- Phase 2 real submission, if later specified, must use `CARTOLA_GLB_TOKEN` from environment or `.env` and must verify expected team identity before POST.
- No CLI command may accept tokens as command arguments.

6.5 Error Handling And Edge Cases

Case	Required Behavior	User/System Impact
Missing historical raw season	Audit classifies the season as incompatible or unavailable with explicit reason.	Operator knows the season cannot support evidence.
Partial current season	Audit labels current-season evidence as partial and non-comparable to complete seasons.	Prevents false promotion evidence.
Missing strict fixture evidence	Strict fixture workflow fails before recommendation or backtest use.	Prevents accidental leakage claims.
Exploratory fixture identity mismatch	Policy simulation or matchup research marks evidence diagnostic-only or fails comparability.	Research cannot promote defaults.
Target live market already finalized	Live recommendation fails unless explicit debug flag is provided; outputs still suppress actuals.	Prevents live output leakage.
No training history before target round	Recommendation or backtest fails with a clear error.	Avoids untrained model output.

Case	Required Behavior	User/System Impact
Infeasible optimized squad	Round output records optimizer status, empty selection, unchanged budget path, and comparability impact.	Failure is visible in reports.
Missing selected-player `variacao` in historical replay	Moving-budget run fails or is marked invalid; missing variation is not treated as zero.	Protects budget-path correctness.
Model candidate has point lift but budget risk	Promotion decision keeps current default until budget guardrails pass.	Prevents risky live default changes.
Confirm submit requested in Phase 1	Command exits with `CONTRACT_UNVERIFIED` before auth or POST construction.	Prevents unverified account mutation.
Secrets in local config	`conf/local` and `.env` content must not be committed or serialized into reports.	Protects account credentials.

7. Quality Attributes

Security And Privacy

- The system must not commit secrets, tokens, local machine config, or authenticated API payloads.
- Phase 1 submission planning must not read `CARTOLA_GLB_TOKEN`.
- Future authenticated submission must verify account/team identity before POST and must not serialize tokens.
- Report artifacts may record source paths, hashes, model configuration, and nonsecret metadata only.
- Bandit checks run against `src/cartola` through the repo quality gate.

Performance And Scale

- Historical moving-budget backtests prioritize correctness over target-round parallelism; target rounds execute sequentially.
- Experiment `--jobs` may control child-run or model-internal behavior but must not imply target-round parallelism for moving-budget child backtests.
- Large experiment and diagnostic outputs stay under `data/08_reporting/` and are referenced by manifests instead of duplicated into trackers.
- Long experiment runs should use environment thread caps from `.env.example` to avoid native thread oversubscription.

Reliability And Operations

- Every research, backtest, recommendation, or submission-planning command must write enough metadata to reproduce source identity, configuration, budget policy, and artifact paths.
- Recommendation run directories must be unique and should never overwrite previous live runs.
- Comparability checks must fail closed when source identity, fixture identity, candidate pools, optimizer status, budget policy, or scoring contract drift.
- Operational live workflows must expose capture age, capture hash, target round, model profile, budget used, formation, captain, and output path.

Accessibility And Localization

- CLI output should remain readable in terminals and CI logs.

- CSV/JSON artifacts are the canonical machine-readable outputs; HTML reports are secondary reviewer artifacts.
- Cartola-facing status normalization must handle Portuguese status labels such as `Provavel` and accented variants when present.
- Times in persisted metadata must use UTC ISO-8601 when they identify run or capture timestamps.

8. Alternatives Considered

Option	Pros	Cons	Decision
Keep fixed-budget backtests as normal evidence	Easier parallelism and comparability with old reports	Does not reflect Cartola patrimonio path and hides budget drawdown risk	Rejected for new evidence
Promote Ridge after 2020-2025 points lead	Higher aggregate historical points in M008	Failed balanced budget-risk gates	Rejected until live budget/risk guardrails exist
Use exploratory fixture reconstruction for strict evidence	More historical coverage	Not pre-lock evidence and can leak schedule knowledge	Rejected for strict claims
Enable real submit in the same phase as submission planning	Reduces manual lineup entry	Authenticated POST contract is unverified and account mutation risk is high	Rejected; Phase 2 requires separate spec
Treat oracle discovery as promotion evidence	Identifies theoretical best squads	Hindsight-only and uses completed-round outcomes	Rejected for default changes

9. Dependencies And Constraints

- **Dependencies:** Python 3.13.12, `uv`, Kedro, pandas, scikit-learn, XGBoost, PuLP, Plotly, Rich, Optuna, Cartola public APIs, TheSportsDB fixture data, local FootyStats CSVs, optional MLflow, GitHub Actions, AgileForge CLI.
- **Constraints:** Moving-budget runs are path-dependent; old fixed-budget artifacts are non-comparable; strict fixture evaluation requires pre-lock fixture evidence; real Cartola submission is disabled; `conf/local` and secrets must remain uncommitted.
- **Assumptions:** The current live default remains `xgboost_depth2_l2_heavy + ppg_xg` until a fresh promotion decision clears budget-risk guardrails. 2026 current-season artifacts are partial until the season completes. Future AgileForge updates will point to this spec or a superseding version.

10. Rollout, Migration, And Compatibility

This is a project-level management spec and does not migrate data by itself.

Compatibility rules:

- New historical evidence must record `budget_policy=moving`.
- Old fixed-budget reports may remain in the repository but must not be mixed into moving-budget rankings.
- Existing feature-level specs under `docs/superpowers/specs/` remain valid references unless superseded by later accepted specs.
- `specs/app.md` is the AgileForge-facing project spec. If this file changes materially, AgileForge project state should be refreshed through an installed spec update flow when available.
- Any future real submission capability requires a separate accepted spec before implementation.

11. Success Metrics

Metric	Target	Measurement Source
Quality gate reproducibility	`uv run --frozen pyrepo-check --all` exits 0 in CI and local verification	CLI output and GitHub Actions
Historical backtest comparability	100% of promotion-candidate runs include matching budget policy, scoring contract, fixture mode, candidate signature, and optimizer status signatures	`comparability_report.json`
Live recommendation traceability	100% of one-command live runs contain live workflow metadata linking capture CSV hash to recommendation artifacts	`live_workflow_metadata.json`
Strict fixture safety	0 strict-mode runs silently fall back to exploratory or no-fixture mode	command failures and metadata audits
Submission safety	100% of Phase 1 `--confirm-submit` invocations fail before auth handling with `CONTRACT_UNVERIFIED`	submission tests and `submission_result.json`
Default promotion discipline	100% of live default changes cite a frozen decision artifact with passed guardrails	promotion decision JSON/Markdown

12. Open Questions

Question	Impact	Owner	Status
What exact live budget/risk guardrails must Ridge or tuned XGBoost clear before default promotion?	Determines whether future point leaders can replace the current XGBoost default.	caRtola maintainers	Open
What verified Cartola save/read-back contract will be accepted for real submission Phase 2?	Blocks authenticated squad submission.	caRtola maintainers	Open
How many strict pre-lock fixture snapshots are required before strict matchup context can be evaluated as promotion evidence?	Determines when matchup context can move beyond research or opt-in live use.	caRtola maintainers	Open
Should legacy notebooks and R scripts be documented as archival only or maintained as supported user workflows?	Affects contributor expectations and documentation scope.	caRtola maintainers	Open
What AgileForge command should be used for future spec updates after project creation?	Determines how this living spec remains synchronized with project management state.	caRtola maintainers	Open

13. Revision History

Date	Version	Change	Author
2026-05-16	0.1	Initial AgileForge-facing project-level technical spec.	Codex